

Development and verification of the neutron diffusion solver for the GeN-Foam multi-physics platform



Carlo Fiorina^{a,*}, Nordine Kerkar^b, Konstantin Mikityuk^c, Pablo Rubiolo^b, Andreas Pautz^{a,c}

^a École polytechnique fédérale de Lausanne, Laboratory for Reactor Physics and Systems Behaviour, 1015 Lausanne, Switzerland

^b LPSC-IN2P3-CNRS/UJF/Grenoble INP, 53 avenue des Martyrs, 38026 Grenoble Cedex, France

^c Paul Scherrer Institut, Nuclear Energy and Safety Department, Laboratory for Reactor Physics and Systems Behaviour, PSI Villigen, 5232, Switzerland

ARTICLE INFO

Article history:

Received 2 February 2016

Received in revised form 13 May 2016

Accepted 23 May 2016

Available online 2 June 2016

Keywords:

Neutron diffusion solver

Finite volumes

OpenFOAM

Discontinuity factors

Solution acceleration

GeN-Foam

ABSTRACT

The Laboratory for Reactor Physics and Systems Behaviour at the PSI and the EPFL has been developing in recent years a new code system for reactor analysis based on OpenFOAM®. The objective is to supplement available legacy codes with a modern tool featuring state-of-the-art characteristics in terms of scalability, programming approach and flexibility. As part of this project, a new solver has been developed for the eigenvalue and transient solution of multi-group diffusion equations. Several features distinguish the developed solver from other available codes, in particular: object oriented programming to ease code modification and maintenance; modern parallel computing capabilities; use of general unstructured meshes; possibility of mesh deformation; cell-wise parametrization of cross-sections; and arbitrary energy group structure. In addition, the solver is integrated into the GeN-Foam multi-physics solver. The general features of the solver and its integration with GeN-Foam have already been presented in previous publications. The present paper describes the diffusion solver in more details and provides an overview of new features recently implemented, including the use of acceleration techniques and discontinuity factors. In addition, a code verification is performed through a comparison with Monte Carlo results for both a thermal and a fast reactor system.

© 2016 Elsevier Ltd. All rights reserved.

1. Introduction

Nuclear reactor analyses are typically performed nowadays using a few legacy codes (see e.g. Lavialle, 2004; GRS, 2012; US NRC, 2001, 2011) that guarantee a high degree of reliability thanks to an extensive process of verification and validation. However these codes suffer from some limitations in terms of core geometric configuration, size of the problem to investigate, and, most notably, possibility of source code modification. This leads to a limited applicability to unconventional reactor designs, as in the case of research reactors or of some innovative reactors developed for instance in the frame of the Generation IV International Forum. In addition, different legacy codes are generally available for different “physics” (e.g., neutronics, thermal-hydraulics, thermal-mechanics) and the relatively old programming paradigms employed in these codes can complicate their integration into modern, non-linearly consistent multi-physics schemes and platforms.

For these reasons, the Laboratory for Reactor Physics and Systems Behaviour (LRS) at the PSI and at the EPFL has started adopting a new development paradigm. Instead of a traditional “top-down” approach aimed at an improved use and coupling of legacy codes, a “bottom-up” approach has been followed where the most modern numerical libraries are used for a quick development *ab initio* of new tools. Thanks to the use of these libraries, the developed solvers inherit modern features in terms of coupling, geometrical flexibility, scalability and programming style, the latter implying easy maintenance and modification of the code.

The use of a bottom-up approach for nuclear code development has been recently made by various researchers, showing promising results. The most notable example is the MOOSE project in the US (Gaston et al., 2009). This project employs the finite element library libMesh (Kirk et al., 2006) to discretize and solve the equations that govern the different phenomena in a nuclear reactor. Legacy codes are not employed except for benchmark purposes or as models to develop modules for specific physics to solve. The coupling between different equations is potentially fully implicit and the use of a Jacobian Free Newton Krylov (JFNK) algorithm is envisaged. Geometrical configurations are made flexible by the use of unstructured meshes. The use of finite elements and a JFNK

* Corresponding author.

E-mail address: carlo.fiorina@epfl.ch (C. Fiorina).

Nomenclature

Latin symbols

C_k	concentration of the k^{th} delayed neutron precursor group [m^{-3}]
$ d $	effective leakage coefficient (Eq. (11)) [m^{-1}]
$\underline{D}_i$	neutron diffusion coefficient for the i^{th} energy group [m]
$\bar{D}_i$	matrix resulting from the finite volume discretization of the diffusion term for the i^{th} energy group [m^{-1}]
H_{AB}	distance between cell center of two adjacent cell A and B [m]
H_{AF}	distance between the cell center of cell A the common face with cell B [m]
H_{FB}	distance between the cell center of cell B the common face with cell A [m]
$\bar{I}$	identity matrix [–]
k_{eff}	effective multiplication factor [–]
L_i	diffusion length for the i^{th} energy group (defined as $L_i^2 = \frac{D_i}{\Sigma_{a,i}}$) [m]
R_i	residual for i^{th} energy group as defined in Eq. (9) [–]
S	cell boundary surface [m^2]
S_d	delayed neutron source [$\text{m}^{-3} \cdot \text{s}^{-1}$]
$S_{n,i}$	fission neutron source from neutron energy groups others than the i^{th} [$\text{m}^{-3} \cdot \text{s}^{-1}$]
$S_{s,i}$	scattering neutron source from neutron energy groups others than the i^{th} [$\text{m}^{-3} \cdot \text{s}^{-1}$]
$ S $	effective source coefficient (Eq. (11)) [m^{-1}]
t	time [s]

u	fuel velocity [$\text{m} \cdot \text{s}^{-1}$]
v_i	average neutron velocity for the i^{th} energy group [$\text{m} \cdot \text{s}^{-1}$]
Vol_c	volume of the c^{th} cell [m^3]

Greek symbols

α	albedo coefficient [–]
β_k	delayed neutron fraction for the k^{th} delayed neutron precursor group [–]
β_t	total delayed neutron fraction [–]
γ_i	field defined in Eq. (14) [s]
λ_k	decay constant for the k^{th} delayed neutron precursor group [s^{-1}]
ν	average number of neutrons per fission [–]
$\Sigma_{a,i}$	absorption cross section for the i^{th} energy group [m^{-1}]
$\Sigma_{r,i}$	removal (disappearance) cross section for the i^{th} energy group [m^{-1}]
$\Sigma_{f,i}$	fission cross section for the i^{th} energy group [m^{-1}]
$\Sigma_{j \rightarrow i}$	group-transfer cross section from the j^{th} to the i^{th} energy group [m^{-1}]
φ_i	neutron flux for the i^{th} energy group [$\text{m}^{-2} \cdot \text{s}^{-1}$]
φ_i^*	neutron flux defined in Eq. (13) for the i^{th} energy group [$\text{m}^{-2} \cdot \text{s}^{-1}$]
$\chi_{d,i}$	delayed neutron yield for the i^{th} energy group [–]
$\chi_{p,i}$	prompt neutron yield for the i^{th} energy group [–]

The cap (as in $\bar{S}_{n,i}$) indicates the discretized form of a variable. The subscript “c” and “f” indicates the value for a certain cell or face. The superscripts “l” and “n” indicates l^{th} intra-step iteration and n^{th} time step, respectively. The apex ‘ indicates a predicted solution. The parentheses || indicate an effective value. “A” and “B” represent two generic adjacent cells.

algorithm for coupling has enormous potential in terms of possible developments and quality of the platform. On the other hand both choices suffer from limited current development, especially in the field of thermal-hydraulics, thus requiring extensive development efforts and time.

Another promising route is the use of the OpenFOAM® library (OpenFOAM, 2016; Weller et al., 1998). Despite its thermal-hydraulic vocation, OpenFOAM is a complete library for the finite-volume discretization and solution of partial differential equations, which makes it suitable for the development of general purpose tools for reactor analysis (Aufiero et al., 2014, 2015; Clifford, 2013; Clifford et al., 2013; Jareteg et al., 2014, 2015). The development strategy in OpenFOAM is based on the use of finite-volume schemes and iterative (Picard) coupling of equations. This poses clear constraints on the possible code developments compared to a finite-element based library with generic coupling strategies. On the other hand, a simple, standardized and robust numerical treatment helps lowering the development efforts and required competences, thus allowing for a wider developer community and for a more effective use of the code as an educational tool. In addition, OpenFOAM is distributed with a significant number of verified and well-performing solvers, especially in the field of thermal-hydraulics. A large number of other solvers and routines are also continuously provided by its user community. The availability of various solvers and routines and the open-source philosophy of the OpenFOAM community constitute formidable tools for favoring code development and avoiding work replicates. For these reasons, OpenFOAM has been chosen as reference library for the development of new tools also by the LRS at the PSI and at the EPFL. The main results of this research effort have been the development of a discrete-ordinate solver for neutron transport

(Aufiero, 2014; Fiorina et al., 2014b), and of a multi-physics tool for steady-state and transient analysis of reactors named GeN-Foam (Fiorina et al., 2015a, 2015b, 2015c). It is worth mentioning that the developed solvers have been fully developed employing the official OpenFOAM release (OpenFOAM, 2016), which distinguishes them from other works e.g. from Clifford and Jasak (2009) and Aufiero (2014), which were instead based on the OpenFOAM Extend platform (OpenFOAM Extend, 2016).

The objectives of this paper are to describe in details the new OpenFOAM-based neutron diffusion solver that has been developed as sub-solver of the GeN-Foam platform; to present its most recent developments; and to verify the solver for core configurations representative of both thermal and fast reactor systems. Section 2 of this paper describes the mathematical model and its implementation. Sections 3 and 4 describe the most recent developments, namely: inclusion of acceleration techniques for the core transient analysis; and the use of discontinuity factors (Smith, 1985), including the possibility to automatically adjust the discontinuity factors based on fluxes from full-core Serpent analysis. Section 5 provides the verification tests performed via comparison with the Monte Carlo code Serpent (Leppänen, 2007) for a mini-core PWR and a sodium-cooled fast reactor. Section 6 presents a preliminary assessment of the acceleration techniques described in Section 3. The conclusions of the work are drawn in Section 7.

2. Solver description

OpenFOAM has been conceived as a C++-based object-oriented library for the discretization and solution of partial differential equations. A neutron diffusion solver shows limited specificities compared to traditional solvers in the field of continuum

mechanics. This allowed to implement the neutron diffusion equations in the form of a standard OpenFOAM solver that: (1) inherits typical OpenFOAM features in terms of parallelization and mesh structure; and (2) can benefit from continuous development of the OpenFOAM base classes, thus maintaining state of the art routines for data manipulation, discretization techniques and matrix solutions.

2.1. Mathematical model

The developed solver is based on the traditional multi-group diffusion equations:

$$\frac{1}{v_i} \frac{\partial \varphi_i}{\partial t} = \nabla D_i \nabla \varphi_i + \frac{v \Sigma_{f,i} (1 - \beta_t) \chi_{p,i}}{k_{eff}} \varphi_i - \Sigma_{r,i} \varphi_i + \frac{S_{n,i} (1 - \beta_t) \chi_{p,i}}{k_{eff}} + S_{d,i} + S_{s,i} \quad (1)$$

where the explicit source terms $S_{n,i}$, $S_{d,i}$ and $S_{s,i}$ are calculated as:

$$S_{n,i} = \sum_{j \neq i} v \Sigma_{f,j} \varphi_j \quad (2)$$

$$S_{d,i} = \sum_k \lambda_k C_k \quad (3)$$

$$S_{s,i} = \sum_{j \neq i} \Sigma_{j \rightarrow i} \varphi_j \quad (4)$$

Different values can be assigned to prompt and delayed neutrons yields. The user can chose whether to use effective delayed neutron fractions and total neutron yields, or physical fractions and differentiated prompt/delayed neutron yields. The precursor concentrations C_k in Eq. (3) are evaluated using the following equations:

$$\frac{\partial C_k}{\partial t} + \nabla \cdot (\mathbf{u} C_k) = \frac{\beta_k \sum_j v \Sigma_{f,j} \varphi_j}{k_{eff}} - \lambda_k C_k \quad (5)$$

Eq. (5) includes a precursor transport term based on a velocity field $\mathbf{u}$, which is essential for neutronics calculations in case of liquid fueled reactors like the molten salt reactors (Fiorina et al., 2014a).

The solver is designed for both time dependent and eigenvalue calculations. In the latter case, the time derivatives in Eqs. (1) and (5) are set to zero. For the time dependent calculations, the k_{eff} can be set to 1 or taken for instance from an eigenvalue calculations to provide an initial steady-state for the reactor. The number and structure of both neutron energy and precursors groups is arbitrary.

2.2. Boundary conditions

OpenFOAM includes a number of built-in boundary conditions. Among these boundary conditions, the zero gradient and the fixed value boundary conditions can conveniently be used in a neutron diffusion solver. In addition, a new boundary condition has been implemented to simulate albedo boundary conditions, i.e., boundary conditions based on the ratio α between incoming and outgoing partial neutron currents. By employing the diffusion theory expression of partial currents (Stacey, 2007), one can demonstrate this condition to be:

$$\frac{D_i}{\varphi_i} \nabla (\varphi_i) = -\frac{1}{2} \left(\frac{1 - \alpha}{1 + \alpha} \right) \quad (6)$$

The albedo α is zero in case of a geometry surrounded by vacuum (zero incoming current). More generally it can be estimated based on the properties of the medium external to the boundary as:

$$\alpha = -\left(\frac{1 - 2D_i/L_i}{1 + 2D_i/L_i} \right) \quad (7)$$

2.3. Cross section parametrization

To assign different cross-sections to different core regions, the mesh can be subdivided into cell zones (i.e., groups of cells) and a cross-section set can be associated to each zone through a dedicated input file. Also the possibility of cross-section parametrization is included in the GeN-Foam diffusion solver. For this purpose, different perturbed cross-section sets can be fed to the solver for each cell zone. In particular, for each cell zone the solver can read up to 8 different cross-section sets for: (1) nominal core conditions; (2) increased/reduced fuel temperature; (3) axially expanded fuel; (4) radially expanded core; (5) increased/reduced coolant density; (6) expanded cladding; (7) increased/reduced coolant temperature; and (8) increased/reduced boron concentration. Each cross section set will also include information about the core conditions referred to nominal and perturbed states (e.g. fuel temperatures associated to the cross sections set for nominal and increased/reduced fuel temperatures). Based on this information, the solver can collect cell-wise information on perturbed variables (temperatures, densities, deformations and boron concentration) from thermal-hydraulics and thermal-mechanic solvers and interpolate the cross sections accordingly, for each cell. A linear interpolation is performed in all cases except for fuel temperatures, for which the interpolation can be based either on the logarithm or on the square root of temperature, based on whether the neutron spectrum is mainly fast or thermal, respectively.

Following the increasing availability of computational resources, it has been decided to privilege the use of cross-section sets prepared via Monte Carlo calculations. To this purpose, an Octave (Eaton et al., 2008) script has been prepared to automatically convert an output from the Monte Carlo code Serpent to an OpenFOAM readable input file. The script can read different cross-section sets from different universes of a full-core or lattice Serpent calculation and associate them to the corresponding cell zones in the OpenFOAM mesh. As an example, in case of full core Serpent calculations, the cross-section preparation for core transient analysis will require 6 Serpent runs for both fast reactors (nominal, increased fuel temperatures, axially and radially expanded fuel, reduced coolant density and expanded cladding) and PWRs (nominal, increased fuel temperatures, reduced coolant density, increased coolant temperature, increased boron concentration). In spite of the availability of a preferential Serpent-based route for cross-section preparation, the cross-section input for the solver is under the form of a simple ASCII file and it allows for a complete freedom on the method for preparing it.

2.4. Mesh deformation and control rod movement

The solver is provided with the possibility to deform the mesh based on a given displacement field to account for the thermal deformation of a core, which allows for an accurate prediction of expansion-related reactivity feedbacks (Fiorina et al., 2015a). In particular, the solver can employ a displacement field with values specified at the cell centers of a generic mesh. It then projects such field onto its own mesh, interpolates from the cell centers to the cell vertices, and uses the resulting vector field to displace the mesh points. Examples of application can be found in (Fiorina et al., 2015a, 2015b).

The possibility has been included to assign to control rods an initial height, as well as to move them during a transient from an initial to a final position at a constant speed. The mesh is not modified but the value of cross sections in each cell is set equal to that of followers or control rods based on the control rod position, which in turns is evaluated based on initial position, speed and initial and final time for control rod movement. No cusping correction has been included to date, but the use of a finite volume approach

typically implies the use of cells of the order of a mean free path for neutrons, thus strongly limiting such effect. In addition, the possibility exists in OpenFOAM to specify control rods as separate and moving mesh regions, with dedicated algorithms for projecting fields between faces of the control rods and the corresponding faces of the main mesh. This would limit the problem of cusping without requiring source code modifications.

2.5. Implementation

Discretization and solution of Eqs. (1) and (5) can be achieved based on the standard finite volume discretization methodology provided by OpenFOAM, which operates on unstructured meshes and optionally includes correctors for non-orthogonal meshes. In particular, an implicit Euler scheme has been used for time integration, so that the discretized form of Eq. (1) can be written as:

$$\begin{aligned} \frac{(\bar{\varphi}_i)^{n+1,l}}{v_i \Delta t} - \bar{D}_i (\bar{\varphi}_i)^{n+1,l} - \bar{I} \frac{v \Sigma_{f,i} (1 - \beta_t) \chi_{p,i}}{k_{eff}} (\bar{\varphi}_i)^{n+1,l} + \bar{I} \Sigma_{r,i} (\bar{\varphi}_i)^{n+1,l} \\ = \frac{(\bar{\varphi}_i)^n}{v_i \Delta t} + (\bar{S}_{n,i} + \bar{S}_{s,i} + \bar{S}_d)^{n,l-1} \end{aligned} \quad (8)$$

where n and $n+1$ indicate the solution for the current time step (for which a solution is available) and for the new time step, respectively. l and $l-1$ indicate instead the solution for the current and previous Picard iterations (within the current time step), respectively.

In case of eigenvalue calculations, a traditional power iteration algorithm (Stacey, 2007) is used and the prompt neutron production term of the i^{th} group in Eq. (8) is left explicit (i.e., calculated from the n^{th} step). In case of transient calculations, the equations for the different energy groups are solved sequentially and the explicit terms are resolved at each time step via Picard iteration.

Residuals are evaluated by OpenFOAM by subtracting right-hand and left-hand sides of Eq. (8) and normalizing based on the source term. This turned out not to be fully adequate to evaluate the convergence of Eq. (8) within a time step in case of transient analyses. The temporal terms in Eq. (8) are in fact small compared to the other terms and the residuals evaluated as difference between right-hand and left-hand sides are dominated by spatial and energy inaccuracies. To properly estimate the degree of convergence of Picard iterations within a time step, the convergence criterion has then been strengthened by an additional condition. In particular, the integral of the variation of fluxes in the last iteration is compared to the total variation in the time step:

$$R_i = \frac{\sum_c Vol_c |(\bar{\varphi}_i)_c^{n+1,l} - (\bar{\varphi}_i)_c^{n+1,l-1}|}{\sum_c Vol_c |(\bar{\varphi}_i)_c^{n+1,l} - (\bar{\varphi}_i)_c^n|} \quad (9)$$

The solver allows the user to define the convergence criteria for both the standard residuals and the residuals defined in Eq. (9), as well as a maximum number of iterations.

2.6. Parallelization

The developed solver has been implemented based on available OpenFOAM classes and programming paradigm. As such, it has inherited the possibility of parallelization based on geometric domain decomposition and MPI protocol. This algorithm has undergone several tests employing standard OpenFOAM solvers, revealing excellent scalability features (OpenFOAM Parallel Computing, 2016). In view of its relatively standard structure, no significant differences in performances are expected for the solver here presented. However, proper scalability tests should be performed and will be the subject of future studies for the authors.

3. Acceleration of the time dependent solution

Eq. (8) generally features an extremely slow convergence rate within each time step. This is due to the structure of the equation and to the magnitude of its terms. For instance, if an instantaneous reactivity insertion is simulated, this will translate into the term $S_{n,i} + S_{s,i} + S_d$ being slightly increased, typically by less than 1%. The overall source term $\frac{(\bar{\varphi}_i)^n}{v_i \Delta t} + (\bar{S}_{n,i} + \bar{S}_{s,i} + \bar{S}_d)^{n,l-1}$ will then be increased by a lower amount (in relative terms), as $\frac{(\bar{\varphi}_i)^n}{v_i \Delta t}$ is constant (calculated from fluxes at the previous time step). Since every term in the left-hand side of Eq. (8) is proportional to the flux, the latter will be increased by the same relative amount as the source term. The source $S_{n,i} + S_{s,i} + S_d$ will then be updated and iteration will continue till the relative change in the fluxes is sufficiently small. Convergence speed thus depends on the ratio between the constant part of the source term $\frac{(\bar{\varphi}_i)^n}{v_i \Delta t}$ and the part which is instead updated at every iteration $(\bar{S}_{n,i} + \bar{S}_{s,i} + \bar{S}_d)^{n,l-1}$. Such a ratio is typically very small in practical situations, which makes the convergence slow, typically demanding tens, or even hundreds of iterations.

A possible solution to the problem would be to couple the equations for the different energy groups in the same matrix, as proposed by Clifford and Jasak (2009). However, the possibility of coupling different equations in the same matrix is not available in the official OpenFOAM release and implementation of such a feature would require relatively deep modification of the OpenFOAM base classes. Several acceleration techniques can be used on segregated systems of equations, such as Chebyshev or variational techniques. Presently, three different algorithms have been implemented in the solver described in this paper. The first one is a traditional Aitken acceleration. This technique is based on the prediction of the solution at the next iteration based on the previous three. In particular, given the solutions at iterations l , $l-1$ and $l-2$, a guess solution can be formulated as:

$$(\bar{\varphi}_i)^{n+1,l} = (\bar{\varphi}_i)^{n+1,l-2} - \frac{((\bar{\varphi}_i)^{n+1,l-1} - (\bar{\varphi}_i)^{n+1,l-2})^2}{(\bar{\varphi}_i)^{n+1,l} - 2(\bar{\varphi}_i)^{n+1,l-1} + (\bar{\varphi}_i)^{n+1,l-2}} \quad (10)$$

The second acceleration technique (from here on referred to as integral predictor) that has been implemented is more “physics based”. It relies on the assumption that spectrum and shape of the fluxes change negligibly within two time steps. The idea is to solve a neutron balance using the integral scalar flux. In practice, equations for the different energy groups (Eq. (8)) are integrated over the volume, summed to each other, and the result divided by the integral flux. This provides effective disappearance cross section $|\Sigma_d|$, effective production term $\left| \frac{v \Sigma_{f,i} (1 - \beta_t) \chi_{p,i}}{k_{eff}} \right|$, effective neutron velocity $|v|$, and suitable coefficients ($|d|$ and $|s|$) that can be multiplied to the integral flux to achieve leakages and source neutrons. With these coefficients, a single scalar equation can be written for the new integral flux Φ^{n+1} as:

$$\begin{aligned} \frac{1}{|v| \Delta t} \Phi^{n+1} - |d| \Phi^{n+1} - \left| \frac{v \Sigma_{f,i} (1 - \beta_t) \chi_{p,i}}{k_{eff}} \right| \Phi^{n+1} + |\Sigma_r| \Phi^{n+1} \\ = \frac{1}{|v| \Delta t} \Phi^n + |s| \Phi^{n+1} \end{aligned} \quad (11)$$

Once the new guess for the integral flux has been achieved, all fluxes are multiplied by its value and divided by the integral flux at the previous time step to achieve a new guess for the fluxes. This operation is performed as a predictor step before the intra-time-step Picard iteration starts.

A third acceleration technique (from here on referred to as implicit predictor) that has been implemented is also based on the introduction of a predictor step, this time consisting of an approximate but fully coupled solution of the system of diffusion equations. This can be achieved by observing that the only off-diagonal term of the matrix equation (8) is the diffusion term $\bar{D}_i(\bar{\varphi}_i)^{n+1,l}$. If this term is evaluated using the flux at previous iteration (while keeping a proportionality to the flux), Eq. (8) becomes diagonal. The problem is thus reduced to a set of algebraic equations, one for each cell and energy group, having the form:

$$\begin{aligned} \frac{(\bar{\varphi}_i)^{n+1}}{v_{i,c}\Delta t} - \frac{v\Sigma_{f,i,c}(1-\beta_t)\chi_{p,i,c}}{k_{eff}}(\bar{\varphi}_i)^{n+1} + \Sigma_{r,i,c}(\bar{\varphi}_i)^{n+1} - \frac{(\bar{D}_i\bar{\varphi}_i)^n}{(\bar{\varphi}_i)^n}(\bar{\varphi}_i)^{n+1} \\ - \sum_{j \neq i} \Sigma_{j \rightarrow i,c}(\bar{\varphi}_j)^{n+1} - \frac{v(1-\beta_t)\chi_{p,i,c}}{k_{eff}} \sum_{j \neq i} v\Sigma_{f,j,c}(\bar{\varphi}_j)^{n+1} \\ = \frac{(\bar{\varphi}_i)^n}{v_{i,c}\Delta t} + (\bar{S}_d)_c^{n+1} \end{aligned} \quad (12)$$

The equations for each energy group referring to the same cell can then be grouped together, thus obtaining for each cell a matrix equation with rang equal to the number of energy groups. These matrix equations can then be solved for each cell to obtain an energy-coupled prediction of the fluxes for the next time step, the only approximation being in the approximated diffusion term.

The effectiveness of the acceleration techniques described above depends strongly on the type of reactor and transient, as well as on the accuracy required. In general terms, the Aitken acceleration significantly reduces the number of required Picard iterations. Clearly, it becomes increasingly effective as the number of iterations required to reach convergence increases, which is the case e.g. when no predictor is used or a high accuracy is required. Conversely, a predictor step is extremely effective in case a low accuracy is required, as the predictor itself is generally capable of reducing the initial residuals to ~ 0.01 .

4. Implementation of discontinuity factors

The possibility to use discontinuity factors (Smith, 1985) has been included in the developed solver. For simplicity, it is assumed that the same discontinuity factor can be used for all faces of a cell-zone (isotropic discontinuity factors). Implementation of isotropic discontinuity factors can be obtained by defining a new field γ_i and by rewriting Eq. (1) as

$$\begin{aligned} \frac{1}{v_i} \frac{\partial \varphi_i}{\partial t} = \nabla \cdot \frac{D_i}{\gamma_i} \nabla \gamma_i \varphi_i + \frac{v\Sigma_{f,i}(1-\beta_t)\chi_{p,i}}{k_{eff}} \varphi_i - \Sigma_{r,i} \varphi_i \\ + \frac{S_{n,i}(1-\beta_t)\chi_{p,i}}{k_{eff}} + S_{d,i} + S_{s,i} \end{aligned} \quad (13)$$

γ_i is defined as uniform inside a cell-zone and discontinuous between cell-zones. It follows that Eq. (13) reverts back to Eq. (1) inside a cell zone, while the discretized neutron current across a face between two cells A and B of different cell zones (Fig. 1) becomes:

$$\left(\frac{D_i}{\gamma_i} \right)_f S_f \frac{\varphi_i^*(A) - \varphi_i^*(B)}{H_{AB}} \quad (14)$$

Using the expression (14) and a harmonic interpolation for achieving $\frac{D_i}{\gamma_i}$ at the cell face, Eq. (14) can be rewritten as:

$$\left(\frac{H_{AF} + H_{FB}}{\frac{H_{AF}}{D_i(A)/\gamma_i(A)} + \frac{H_{FB}}{D_i(B)/\gamma_i(B)}} \right)_f S_f \frac{\gamma_i(A)\varphi_i(A) - \gamma_i(B)\varphi_i(B)}{H_{AB}} \quad (15)$$

which is equivalent to a traditional definition of current in a diffusion code employing two discontinuity factors $\gamma_i(A)$ and $\gamma_i(B)$ on the two sides of a face (Smith, 1985).

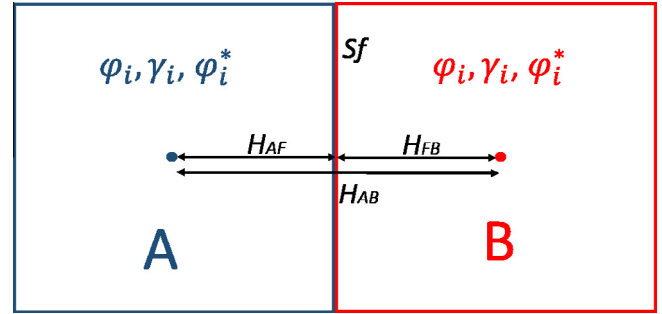


Fig. 1. Schematic representation of two cells belonging to different cell zones.

To solve Eq. (13), a continuous flux φ_i^* can be defined as (Fig. 2):

$$\varphi_i^* = \gamma_i \varphi_i \quad (16)$$

and Eq. (13) can then be rewritten as:

$$\begin{aligned} \frac{1}{\gamma_i v_i} \frac{\partial \varphi_i^*}{\partial t} = \nabla \cdot \frac{D_i}{\gamma_i} \nabla \varphi_i^* + \frac{v\Sigma_{f,i}(1-\beta_t)\chi_{p,i}}{k_{eff}\gamma_i} \varphi_i^* - \frac{\Sigma_{r,i}\varphi_i^*}{\gamma_i} \\ + \frac{S_{n,i}(1-\beta_t)\chi_{p,i}}{k_{eff}\gamma_i} + S_{d,i} + S_{s,i} \end{aligned} \quad (17)$$

and solved for the continuous flux φ_i^* . The discontinuous flux φ_i can then be obtained using Eq. (16).

The method employed to implement discontinuity factors has the disadvantage of allowing only for isotropic discontinuity factors. This limitation is partly compensated by the complete flexibility that the solver developed gives in terms of meshing and mesh subdivision in cell zones. It is for instance possible to refine the definition of cell zones (e.g. to a pin level) in regions of particular interest (domain decomposition approach).

The possible use of cross-section sets prepared via full-core analysis (see Section 2.3) offers the interesting possibility of using the integral of the flux over a core region as predicted by Serpent to adjust γ_i in the corresponding cell zone in OpenFOAM. In such a way, it becomes possible to exactly reproduce in the diffusion code the same integral reaction rates that Serpent predicts for each cell zone. It should be noted however that this does not guarantee a correct prediction of the flux shape inside a cell zone.

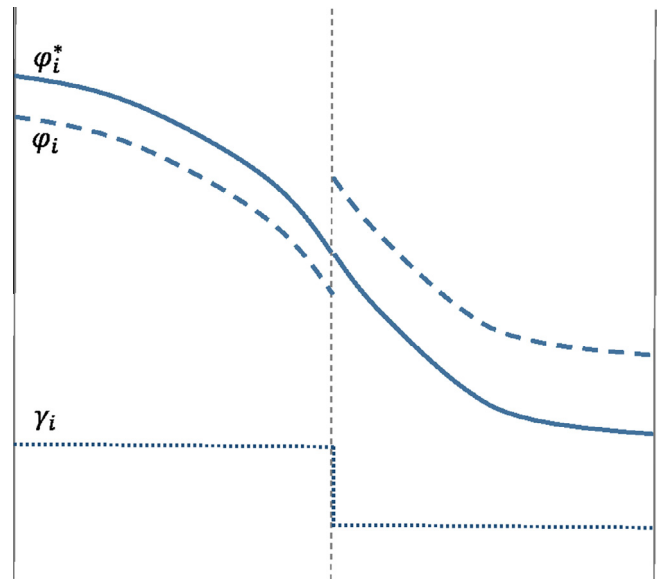


Fig. 2. Schematic exemplification of Eq. (16).

The procedure to automatically adjust the discontinuity factors is carried out in the developed solver as follows:

- Cross sections are generated using Serpent by subdividing the core into regions (universes) that corresponds exactly to the cell zones in the OpenFOAM mesh;
- The integral of each flux over each region is extracted from the Serpent output;
- At each iteration during an eigenvalue calculations, γ_i is evaluated in each cell zone as the ratio between the zone-wise integral fluxes predicted by Serpent and those predicted by OpenFOAM.

Such a procedure has been included as optional in the developed solver, while keeping the possibility to feed the solver with discontinuity factors previously calculated with other codes. It is worth mentioning that the adjustment of discontinuity factors requires a full-core Monte Carlo analysis, implying that such procedure becomes of interest only when employing the solver e.g. for transient analyses.

It is worth noting that the implementation of discontinuity factors and their adjustment described above has some similarities with the superhomogenization (SPH) method described e.g. in (Grundmann and Mittag, 2011; Nikitin et al., 2015). In particular, in the SPH method, a superhomogenization correction factor is defined similarly to γ_i in the adjustment procedure described above. However, while in the solver here presented this correction factor is applied to the flux, in the SPH method it is applied to cross-sections.

5. Solver verification

To verify its performances and accuracy, the solver has been applied to the analysis of two different reactor configurations, namely: a PWR mini-core (Sjenitzer et al., 2015) derived from the OECD/NEA benchmark presented by Kozłowski and Downar (2003); and the European Sodium Fast Reactor (Fiorini and Vasile, 2011). The results are compared with calculations performed using the Serpent Monte Carlo code. The verification is limited to eigenvalue calculations as verification of time integration is provided in (Fiorina et al., 2015a, 2015b). In all cases, a second order discretization is obtained based on a Gauss discretization scheme with harmonic interpolation of diffusion coefficients. The mesh convergence has been verified via progressive refinement until further doubling of the mesh size in each direction would change the k_{eff} prediction by a few pcm at most, which is consistent with the uncertainty of the Serpent calculations.

5.1. PWR mini-core

The PWR mini-core benchmark consists of a group of 9 fuel assemblies: a central UO₂ assembly surrounded by 8 MOX assemblies. A water reflector with a thickness equal to that of the assemblies is placed radially around the 9 fuel assemblies. A control rod is inserted in the upper part of the UO₂ central assembly. Fig. 3 shows a cut of the geometry, while details about dimensions and materials can be found in Sjenitzer et al. (2015).

A mesh sensitivity analysis has shown that a regular hexahedral mesh with a radial discretization equal to half the pin pitch (four cells per pin) allows achieving mesh independence with a second order discretization scheme, leading to approximately 15 million cells. Such a fine discretization is made necessary by the strong gradients caused by the presence of a control rod, and by the steep gradients and high leakages at the axial boundaries. The computational time is significant, of the order of several minutes with 16

cores, which confirms the well-known limitations of the use of a fully unstructured mesh in case of “single-physics” calculations for standard core geometries.

Few-groups cross sections have been generated based on full core Serpent calculations, with universes defined as in Fig. 3, namely: water reflector; corner MOX assemblies in the rodged part; side MOX assemblies in the rodged part; central UO₂ assembly in the rodged part; corner MOX assemblies in the unrodged part; side MOX assemblies in the unrodged part; and central UO₂ assembly in the unrodged part. Serpent calculations predict multiplication factors equal to $1.01345 \pm 3.8 \cdot 10^{-5}$ and $0.99992 \pm 3.8 \cdot 10^{-5}$ for reflective and void boundary conditions, respectively.

Eigenvalue calculations have been performed with and without the automatic adjustment of discontinuity factors described in Section 4. The computations have been performed using 1, 2 or 8 energy groups and both void and reflective boundary conditions. The 8-groups energy structure has been taken from the CASMO-4 code (Edenius et al., 1995). In OpenFOAM, void conditions have been simulated using albedo boundary conditions (Section 2.2) with an albedo coefficient α equal to zero. Table 1 summarizes the obtained results in terms of reactivity difference compared to Serpent (DF indicates the discontinuity factors).

Diffusion calculations without discontinuity factors lead to discrepancies in the multiplication factors of the order of 1000–2000 pcm. In particular, one-group calculations tend to overestimate the k_{eff} , while 2- and 8-groups calculations lead to lower multiplication factors. The leakage component that is present in case of void boundary conditions gives a negative contributions to discrepancies, implying an overestimation of leakages from diffusion calculations. As expected, such an overestimation reduces with increasing number of energy groups.

The use of adjusted discontinuity factors guarantees for each mesh zone the same reaction rates as in Serpent. In the case of reflective boundaries this is expected to lead to the same multiplication factor, as confirmed by results in Table 1. In case of void boundary conditions, discrepancies with Serpent are caused once again by an overestimation of leakages. It is worth noting that such leakage component to the overall discrepancy appears to be higher when discontinuity factors are used.

Figs. 4–6 present radial and axial flux distributions obtained from Serpent and 2-groups OpenFOAM calculations in case of void boundary conditions, both with and without use of discontinuity factors. The uncertainty of the Serpent flux distribution is negligible, of the order of 0.1%. It is interesting to note that the use of automatically adjusted discontinuity factors notably improves the predictive capabilities of the diffusion solver also in terms of flux profiles. Fig. 6 also shows that the use of discontinuity factors causes a steeper gradient at the bottom of the core, which is consistent with the higher over-prediction of leakages previously pointed out. A higher flux is observed in the unrodged region for Serpent and OpenFOAM with discontinuity factors, consistently with the higher predicted reactivity.

5.2. ESFR – European Sodium Fast Reactor

The ESFR (Figs. 7 and 8) is a 3.6 GW_{th} pool-type sodium-cooled fast reactor. It has been designed during the EURATOM ESFR project within the seventh European Framework Program (FP7) and design details can be found in references Fiorini and Vasile (2011) and Lázaro et al. (2014). A slightly modified and simplified design has been used in this work that features a 60° symmetry in the core configuration (Fig. 7b). More details can be found in (Fiorina et al., 2015a, 2015b).

A mesh sensitivity analysis has been performed, showing that mesh independence is achieved by radially subdividing each assembly in 24 triangles. This leads to approximately 1.5 million

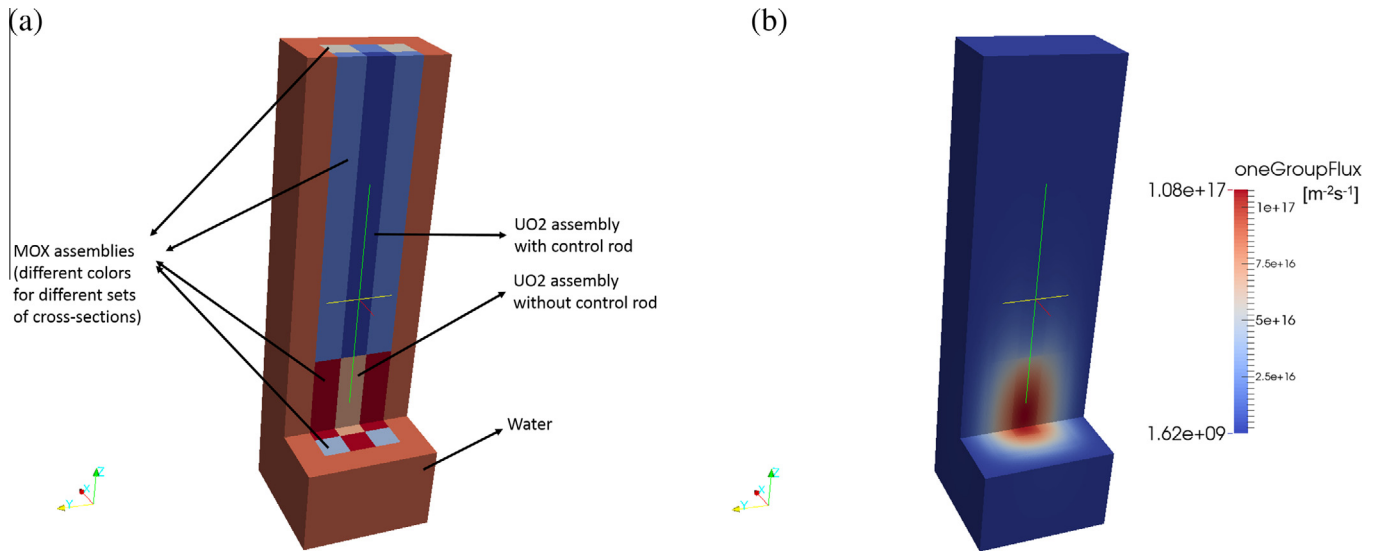


Fig. 3. (a) Geometry of the mini-core PWR benchmark, and (b) flux distribution predicted by a two-groups OpenFOAM calculation with automatic adjustment of discontinuity factors.

Table 1

Reactivity difference between OpenFOAM and Serpent for the PWR mini-core (pcm).

	Reflective boundary	Void boundary [*]
OpenFOAM 1-group with DF	1	−616
OpenFOAM 1-group without DF	2281	2133
OpenFOAM 2-group with DF	−9	−233
OpenFOAM 2-group without DF	−1307	−1366
OpenFOAM 8-group with DF	−5	−134
OpenFOAM 8-group without DF	−1475	−1536

^{*} Simulated using albedo boundary conditions in OpenFOAM.

cells, and a 12-core computational time for an eigenvalue calculation of the order of 1 to several minutes.

Few-groups cross sections have been generated based on full core Serpent calculations, with universes defined as in Fig. 7, namely: lower gas plenum, lower reflector, inner core, outer core, upper gas plenum, upper reflector, radial reflector, central assembly, followers and control rods. Serpent calculations predict a multiplication factor equal to $1.02047 \pm 3.8 \cdot 10^{-5}$.

Eigenvalue calculations have been performed with or without the automatic adjustment of discontinuity factors, using 1 or 24 energy groups. In this case, only void boundary conditions have been considered. Table 2 summarizes the obtained results in terms of reactivity differences compared to Serpent.

Standard diffusion calculations provide discrepancies in multiplication factors of a few hundreds of pcm compared to Serpent. In particular, the 24-groups calculation provides a multiplication factor ~ 400 pcm lower, which is consistent with results obtained with the ERANOS diffusion and transport solvers (Rimpault et al., 2002) for a similar core configuration (Pelloni et al., 2011).

Use of adjusted discontinuity factors notably improves the code predictive capabilities. As discussed, when adjusting the discontinuity factors based on Serpent reaction rates, discrepancies in multiplication factors can only be related to leakages. Such discrepancies can be limited by employing sufficiently large geometries, and by increasing the number of energy groups. Table 2 shows that a 22 pcm discrepancy can be achieved in the case here considered by using 24 energy groups. Also in this case, use of

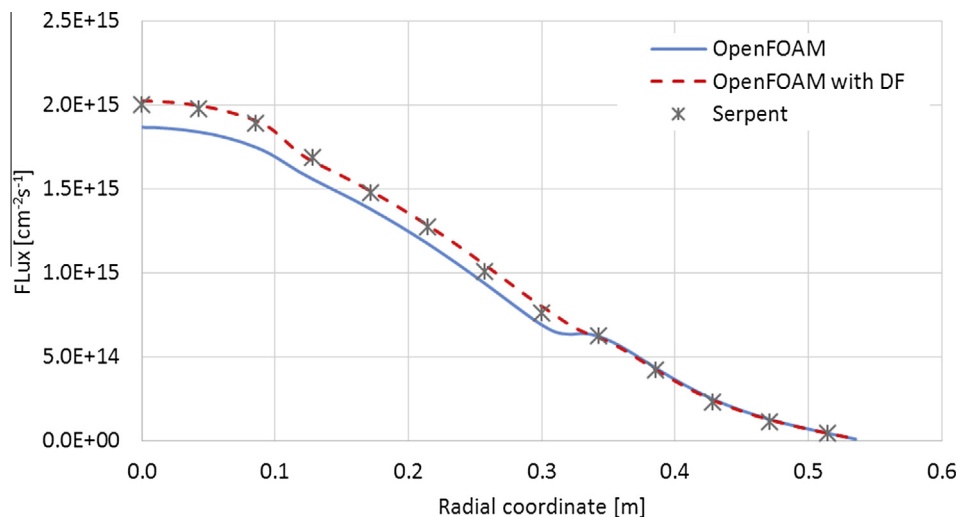


Fig. 4. Radial profile (along x axis) of the mini-core PWR at approximately 96 cm from the core bottom.

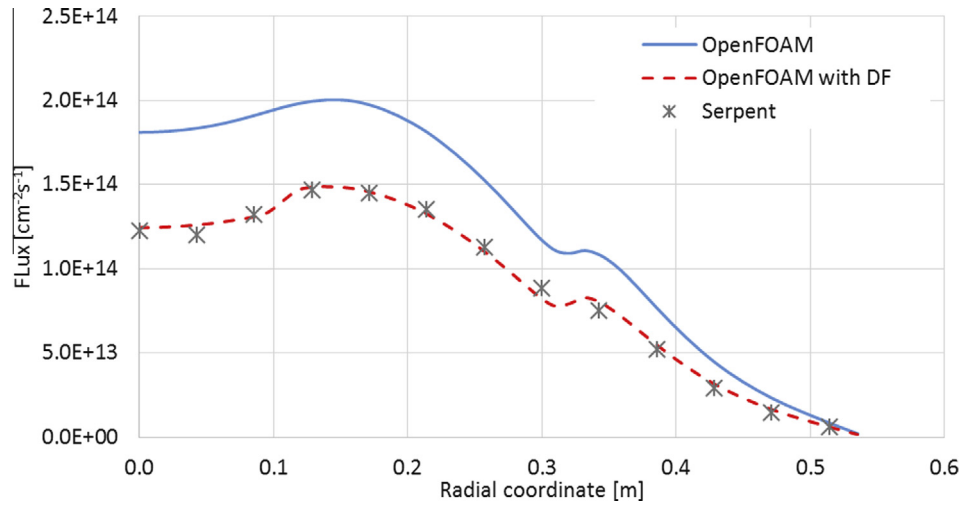


Fig. 5. Radial profile (along x axis) of the mini-core PWR at mid core.

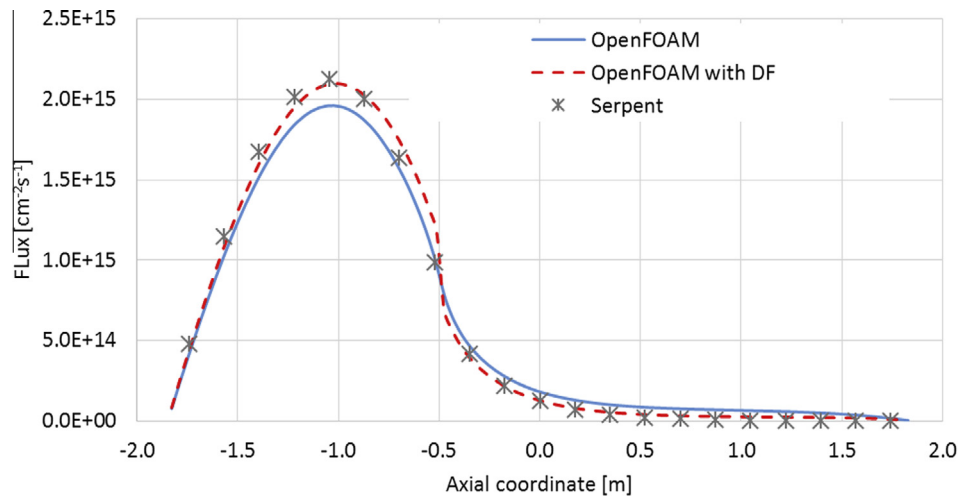


Fig. 6. Axial profile of the mini-core PWR at centerline.

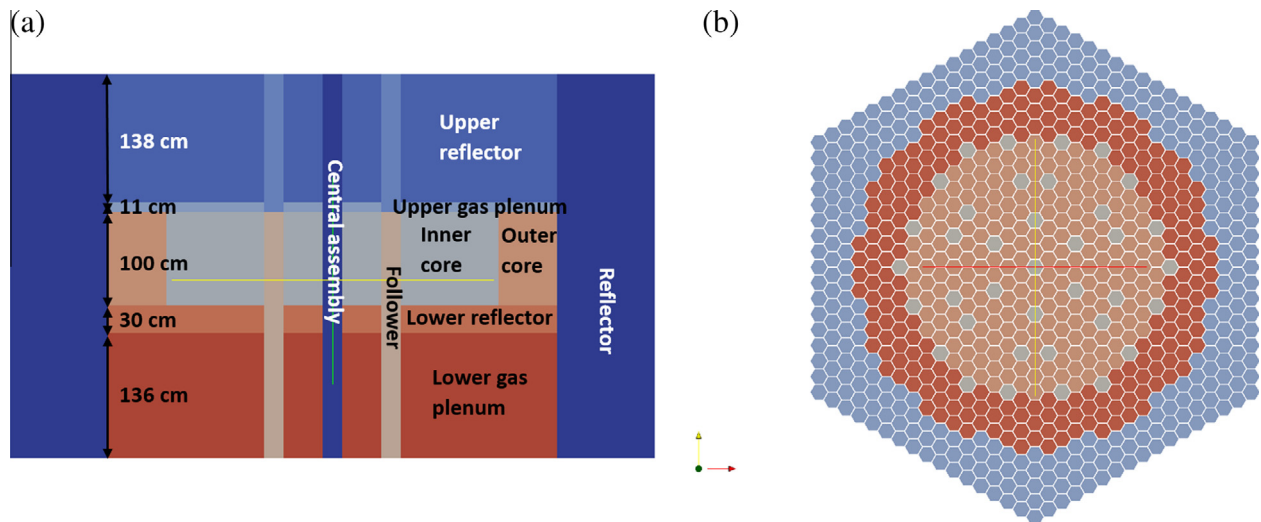


Fig. 7. Axial (a) and radial (b) configurations of the reference ESRF core.

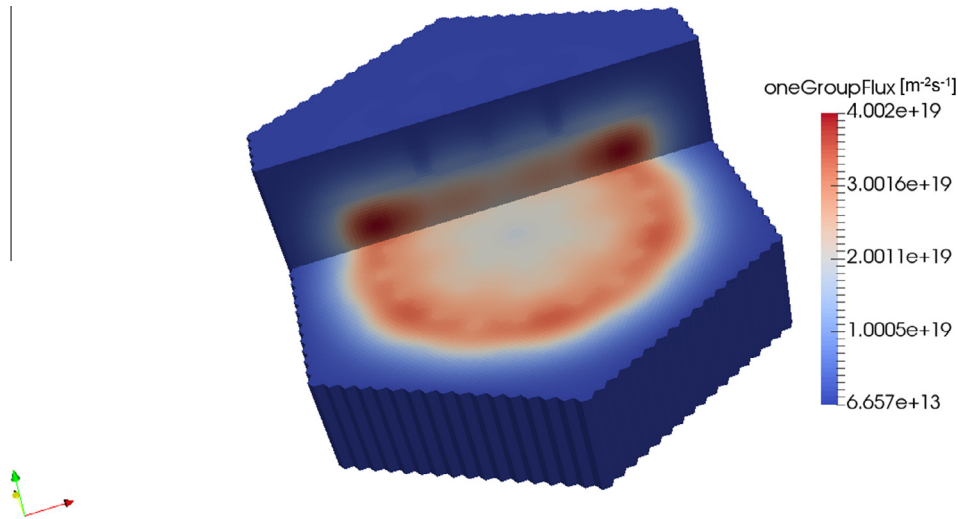


Fig. 8. Flux distribution predicted for the ESRF by a 24-groups OpenFOAM calculation with automatic adjustment of discontinuity factors.

Table 2

Reactivity difference between OpenFOAM and Serpent for the ESRF (pcm).

OpenFOAM 1-group with DF	−253
OpenFOAM 1-group without DF	522
OpenFOAM 24-group with DF	−22
OpenFOAM 24-group without DF	−371

adjusted discontinuity factors notably improves the prediction of flux distributions, as shown in Figs. 9 and 10. Also in this case, the uncertainty of the Serpent flux distribution is of the order of 0.1%.

6. Preliminary assessment of the effectiveness of acceleration techniques

The acceleration techniques described in Section 3 have been applied to two different transients, both for the mini-core PWR and for the ESRF. In particular ~ 700 pcm super-prompt-critical reactivity excursions have been simulated both through a uniform variation of the fission cross-section, and through extraction of the

control rods, the second transient implying a significant flux distortion. In all cases, ten time steps have been simulated, each one determining a change of power of approximately 3%. Tolerance of 10^{-3} and 10^{-7} have been imposed to the residual defined in Eq. (9) and to the standard OpenFOAM residuals, respectively. Results are shown in Table 3.

A significant speed-up is observed in all cases. The Aitken acceleration appears to be comparable or slightly less effective than the two predictors, but it is worth noticing that its effectiveness grows with the number of iterations required. For instance, if the same transient is performed employing 10 times longer time steps or 10 times smaller residuals, more than 100 iterations are required if no acceleration techniques are employed, while the Aitken acceleration limits the iterations to ~ 20 . The 2 predictors have been confirmed to generate a first initial residual (according to Eq. (9)) approximately equal to 10^{-2} , which lowers by two orders of magnitude the residual reduction that must be achieved via Picard iteration. As mentioned in Section 3, the effectiveness of a predictor step strongly depends on the required degree of accuracy, since a small accuracy would require a very few iterations to reach convergence after a predictor step while an extremely high accuracy

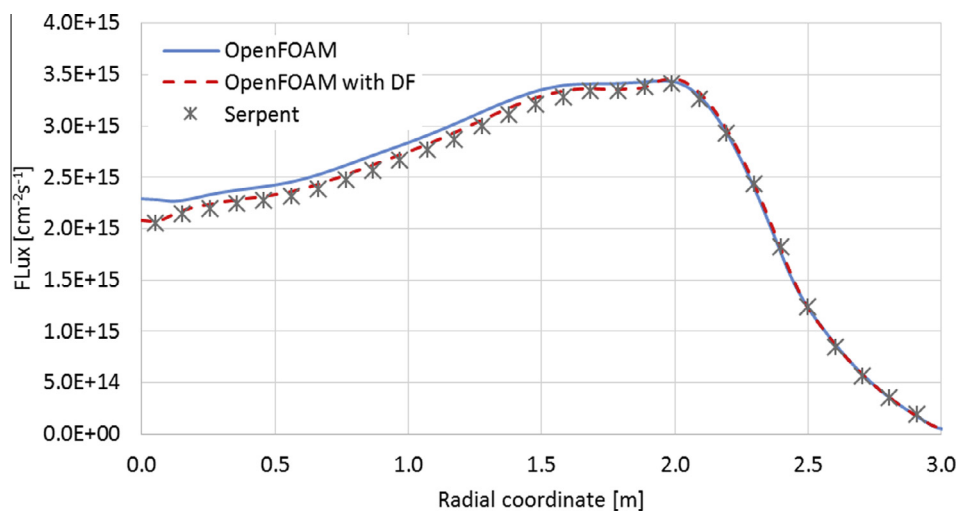


Fig. 9. Radial profile (along x axis) of the ESRF at mid core.

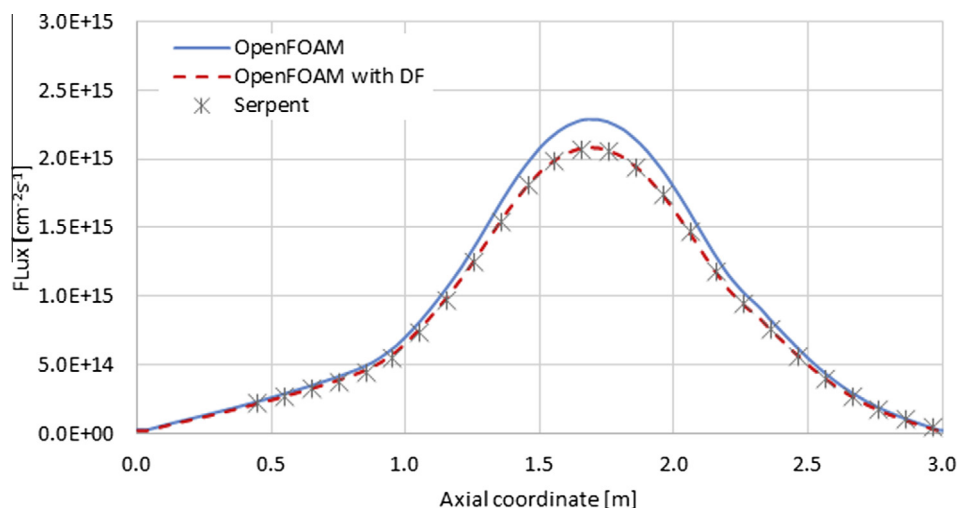


Fig. 10. Axial profile of the ESFR at centerline.

Table 3

Average number of Picard iterations for different acceleration techniques and considering a ~ 700 pcm reactivity insertion for the mini-core PWR and for the ESFR.

		No acceleration	Aitken acceleration	Integral predictor	Implicit predictor
Mini-core PWR (2 energy groups)	Increased fission cross section	23	18	12	6
	Extracted control rod (0.5 m)	23	16	14	13
ESFR (27 energy groups)	Increased fission cross section	22	14	20	9
	Extracted control rod (0.25 m)	21	16	17	9

would make an initial rough prediction mostly ineffective. A better performance is generally observed for the implicit predictor vs the integral predictor. This is particularly the case for the ESFR. The main reason for this is that the integral predictor changes each flux by the same factor, which has been observed to trigger some oscillations in the subsequent Picard iterations.

Results provided in this Section are meant to provide a first insight into the effectiveness of the acceleration techniques implemented in the GeN-Foam diffusion solver. An in-depth investigation of the effectiveness of the different acceleration techniques should involve simulation of different transients for different reactors, which will be the subject of future studies for the authors.

7. Conclusions

A new multi-group neutron diffusion tool has been developed at the PSI and the EPFL for eigenvalue and transient analysis. The solver is based on OpenFOAM and it is integrated into the GeN-Foam multi-physics platform (Fiorina et al., 2015a, 2015b, 2015c). It presents modern features in terms of parallel computing capabilities, use of general unstructured meshes, possibility of mesh deformation, cell-wise parametrization of cross-sections, and arbitrary energy group structure.

The GeN-Foam neutron diffusion solver has been presented in details in this paper, including the underlying mathematical model, available boundary conditions, cross section preparation and parametrization, and features in terms of mesh deformation and control rods movement. Implementation aspects have also been discussed, including the proposal and implementation of new residuals for transient analysis.

Three different acceleration techniques have been presented, allowing to drastically reduce computational times for transient analysis. In particular, a standard Aitken acceleration has been implemented to limit the number of Picard iterations necessary to achieve a desired residual reduction. In addition to the Aitken

acceleration, two different predictor steps have been implemented to lower the first iteration residual. A first predictor is based on an integral neutron balance while the second predictor is based on a cell-by-cell implicit coupling of the diagonalized multi-group diffusion equations. Both predictors can reduce the initial iteration residual from 1 to $\sim 10^{-2}$, with the implicit predictor having shown better performances for the cases analyzed in the paper.

Isotropic discontinuity factors have been implemented, including a procedure to automatically adjust their value to replicate the same reaction rates predicted by a full core Serpent calculation for each region (universe) employed to generate the cross-sections. Such procedure is clearly beneficial in terms of reactivity prediction, as reactivity differences compared to Serpent will only be caused by an erroneous prediction of leakages. In addition, use of adjusted discontinuity factors has shown notably improved predictive capabilities in terms of local flux shape, at least in the cases here investigated.

The developed solver has been verified against Serpent results for both a sodium fast reactor and a 9-assembly PWR mini-core benchmark. Good agreement is observed in both cases, with discrepancies that can be ascribed to the diffusion approximation, and that tend to disappear when adjusted discontinuity factors are employed.

Future work on the neutronics solver for GeN-Foam will include extension to SP_3 methodology, parametrization of cross-section based on Xenon content, and further verification and validation, including transient analyses. The possibility to include anisotropic discontinuity factors will also be considered.

Acknowledgements

The first author would like to personally thank Dr. Ivor Clifford (PSI) for his help in the development of the GeN-Foam platform, and acknowledges the support of the OpenFOAM nuclear special interest group (http://openfoamwiki.net/index.php/Sig_Nuclear).

3-D meshes for the ESFR have been generated using Gmsh (Geuzaine and Remacle, 2009). 3-D meshes for the PWR mini-core have been generated using Salome (Salome, 2016).

References

- Aufiero, M., 2014. Development of Advanced Simulation Tools for Circulating Fuel Nuclear Reactors (PhD thesis). Politecnico di Milano, Italy.
- Aufiero, M., Cammi, A., Geoffroy, O., Losa, M., Luzzi, L., Ricotti, M.E., Rouch, H., 2014. Development of an OpenFOAM model for the molten salt fast reactor transient analysis. *Chem. Eng. Sci.* 111, 390–401.
- Aufiero, M., Fiorina, C., Laureau, A., Rubiolo, P., Valtavirta, V., 2015. Serpent-OpenFOAM coupling in transient mode: simulation of a godiva prompt critical burst. In: Proceedings of ANS MC2015 – Joint International Conference on Mathematics and Computation (M&C), Supercomputing in Nuclear Applications (SNA) and the Monte Carlo (MC) Method, Nashville, Tennessee, April 19–23, 2015.
- Clifford, I., 2013. A Hybrid Coarse and Fine Mesh Solution Method for Prismatic High Temperature Gas-cooled Reactor Thermal-fluid Analysis (PhD thesis). PennState, US.
- Clifford, I., Jasak, H., 2009. Application of multi-physics toolkit to spatial reactor dynamics. In: International Conference on Advances in Mathematics, Computational Methods, and Reactor Physics (M&C), Saratoga Springs, NY, USA, May 3–7 2009.
- Clifford, I., Ivanov, K.N., Avramova, M.N., 2013. A multi-scale homogenization and reconstruction approach for solid material temperature calculations in prismatic high temperature reactor cores. *Nucl. Eng. Des.* 256, 1–13.
- Eaton, J.W., Bateman, D., Hauberg, S., 2008. GNU Octave Manual Version 3.
- Edenius, M., Ekberg, K., Forssen, B.H., Knott, D., 1995. CASMO-4. A fuel Assembly Burnup Program. User's Manual. Studsvik Report SOA-95/1, Studsvik of America.
- Fiorina, C., Lathouwers, D., Aufiero, M., Cammi, A., Guerrieri, C., Kloosterman, J.L., Luzzi, L., Ricotti, M.E., 2014a. Modelling and analysis of the MSFR transient behavior. *Ann. Nucl. Energy* 64, 485–498.
- Fiorina, C., Aufiero, M., Pelloni, S., Mikityuk, K., 2014b. A time-dependent solver for coupled neutron-transport thermal-mechanics calculations and simulation of the Godiva prompt-critical bursts, ICONE-22 Conference, July 7–11, Prague, Czech Republic.
- Fiorina, C., Mikityuk, K., 2015a. Application of the new GeN-Foam multi-physics solver to the European sodium fast reactor and verification against available codes, ICAPP 2015 Conference, May 03–06, Nice, France.
- Fiorina, C., Clifford, I.D., Aufiero, M., Mikityuk, K., 2015b. GeN-Foam: a novel OpenFOAM® based multi-physics solver for 2D/3D transient analysis of nuclear reactors. *Nucl. Eng. Des.* 294, 24–37.
- Fiorina, C., Mikityuk, K., Pautz, A., 2015c. GeN-Foam: a novel multi-physics solver for reactor analysis – Status and ongoing developments. In: ANS Winter Meeting, November 8–12, 2015, Washington DC, US.
- Fiorini, G.L., Vasile, A., 2011. European Commission – 7th Framework Program, “The collaborative project on european sodium fast reactor (CP-ESFR)”. *Nucl. Eng. Des.* 241, 3461–3469.
- Gaston, D., Newman, C., Hansen, G., Lebrun-Grandié, D., 2009. MOOSE: A parallel computational framework for coupled systems of nonlinear equations. *Nucl. Eng. Des.* 239, 1768–1778.
- Geuzaine, C., Remacle, J.-F., 2009. Gmsh: a three-dimensional finite element mesh generator with built-in pre- and post-processing facilities. *Int. J. Numer. Meth. Eng.* 79, 1309–1331.
- GRS, 2012. ATHLET Mod 3.0 Cycle A: Program Overview. Gesellschaft für Anlagen- und Reaktorsicherheit mbH, Germany.
- Grundmann, U., Mittag, S., 2011. Super-homogenisation factors in pinwise calculations by the reactor dynamics code DYN3D. *Ann. Nucl. Energy* 38, 2111–2119.
- Jareteg, K., Vinai, P., Demazière, C., 2014. Fine-mesh deterministic modeling of PWR fuel assemblies: Proof-of-principle of coupled neutronic/thermal-hydraulic calculation. *Ann. Nucl. Energy* 68, 247–256. <http://dx.doi.org/10.1016/j.anucene.2013.12.019>.
- Jareteg, K., Vinai, P., Sasic, S., Demazière, C., 2015. Coupled fine-mesh neutronics and thermal-hydraulics – Modeling and implementation for PWR fuel assemblies. *Ann. Nucl. Energy* 84, 244–257.
- Kirk, B.S., Peterson, J.W., Stogner, R.H., Carey, G.F., 2006. libMesh: a C++ library for parallel adaptive mesh refinement/coarsening simulations. *Eng. Comput.* 22 (3–4), 237–254.
- Kozłowski, T., Downar, T., 2003. OECD/NEA and US NRC PWR MOX/UO₂ core transient benchmark.
- Lavialle, G., 2004. The CATHARE 2 V2.5 code: Main features. In: CATHARE-NEPTUNE International Seminar, May 10–12, 2004, Grenoble, France.
- Lázaro, A., Ammirabile, L., Bandini, G., Darmet, G., Massara, S., Dufour, Ph., Tosello, A., Gallego, E., Jiménez, G., Mikityuk, K., Schikorr, M., Bubelis, E., Ponomarev, A., Krussmann, R., Stempniewicz, M., 2014. Code assessment and modelling for design basis accident analysis of the European sodium fast reactor design. Part I: System description, modelling and benchmarking. *Nucl. Eng. Des.* 266, 1–16.
- Leppänen, J., 2007. Development of a New Monte Carlo Reactor Physics Code (D.Sc. thesis). Helsinki University of Technology, Finland.
- Nikitin, E., Fridman, E., Mikityuk, K., 2015. On the use of the SPH method in nodal diffusion analyses of SFR cores. *Ann. Nucl. Energy* 85, 544–551.
- OpenFOAM, 2016. <<http://www.openfoam.org/>>.
- OpenFOAM Extend, OpenFOAM® Extend Project, 2016. <<http://www.extend-project.de/>>.
- OpenFOAM Parallel Computing, 2016. <<http://www.openfoam.org/features/parallel-computing.php>>.
- Pelloni, S., Mikityuk, K., Rimpault, G., 2011. Nuclear data uncertainty analysis for the Generation IV Sodium-cooled Fast Reactor core with fresh oxide fuel. In: Proc. Int. Conf. ICAPP2011, Nice, France, May 2–5, 2011.
- Rimpault, G., Plisson, D., Tommasi, J., Jacqmin, R., Rieunier, J., Verrier, D., Biron, D., 2002. The ERANOS code and data system for fast reactor neutronic analyses. In: Proc. Int. Conf. PHYSOR 2002, Seoul, Korea, October 7–10, 2002.
- Salome, 2016. <<http://www.salome-platform.org/>>.
- Sjenitzer, N.L., Hoogenboom, J.E., Jiménez Escalante, J., Sanchez Espinoza, V., 2015. Coupling of dynamic Monte Carlo with thermal-hydraulic feedback. *Ann. Nucl. Energy* 76, 27–39.
- Smith, K.S., 1985. Assembly homogenization techniques for light water reactor analysis. *Prog. Nucl. Energy* 17, 303–335.
- Stacey, W.M., 2007. Nuclear Reactor Physics. WILEY-VCH Verlag GmbH & Co. KGaA.
- US NRC, 2001. Relap5/Mod3.3 Code Manual Volume I: Code Structure, System Models, and Solution Methods. US Nuclear Regulatory Commission, December 2001.
- US NRC, 2011. TRACE V5.435 Theory Manual, Field Equations, Solution Methods and Physical Models. US Nuclear Regulatory Commission, January 2011.
- Weller, H.G., Tabor, G., Jasak, H., Fureby, C., 1998. A tensorial approach to computational continuum mechanics using object-oriented techniques. *Comput. Phys.* 12, 620–631.